MILVUS NATIVE MULTISEARCH · PROXY OWNERSHIP REFACTOR · 1 NOVEMBER 2026

One Proxy, one existing Search execution path

Server.MultiSearch now delegates to Proxy.MultiSearch. The separate MultiSearchExecutor type and its Server field have been removed. The existing DQL scheduler still executes every child search.

> **What changed:** batch coordination and shared admission state now belong to the existing Proxy. The shared Search entry callback terminates directly at Proxy.Search. There is no child call back through Server.Search, no new serving endpoint, and no replacement search worker pool.

---

1. Integration: thin transport, Proxy-owned coordination

The public service and Python method are unchanged: MilvusClient.multi_search sends one MilvusService.MultiSearch RPC containing independent SearchRequest items. Each item chooses its own database, collection and parameters; results remain independent and ordered.

```
func (s *Server) MultiSearch(ctx context.Context, req *milvuspb.MultiSearchRequest) (*milvuspb.MultiSearchResponse, error) {
    return s.proxy.MultiSearch(ctx, req)
}
```

> Existing gRPC Server → Proxy.MultiSearch

> Validate envelope · acquire batch permit · bounded dispatch

> Each child: shared Search entry checks → Proxy.Search

> NewSearchTask → existing DqQueue → existing worker pool

> PreExecute → QueryNode shard calls → PostExecute

> Independent child outcomes → one ordered response

The parent and its bounded dispatch goroutines stay outside the DQL worker pool. Only existing SearchTasks enter the queue. Waiting parks Go goroutines but retains state; executing tasks still occupy logical pool slots while waiting for QueryNodes. Admission limits bound pressure without establishing priority or reserving ordinary-Search capacity.

The new state lives on the existing Proxy:

```
// Fields on the existing Proxy, beside its existing scheduler.
sched *scheduler.TaskScheduler
multiSearchInit sync.Once
multiSearchLimits multiSearchLimits
multiSearchBatches chan struct{}
multiSearchActive chan struct{}
multiSearchEntry atomic.Pointer[multiSearchEntry]
```

2. Keep ordinary Search entry checks

Calling Proxy.Search directly does not run gRPC interceptors. During startup, the transport installs the same Search entry chain used by ordinary RPCs. Its terminal handler calls the existing Proxy component directly. The chain preserves database normalization, authentication, per-collection RBAC, quota checks, hooks and independent request statistics.

```
func newMultiSearchEntry(component types.ProxyComponent, limiter types.Limiter, getMetaCache func() proxy.Cache)
func(context.Context, *milvuspb.SearchRequest) (*milvuspb.SearchResults, error) {
    entry := proxyEntryInterceptor(limiter, getMetaCache)
    return func(ctx context.Context, req *milvuspb.SearchRequest) (*milvuspb.SearchResults, error) {
        // Fork mutable stats; the access-log interceptor similarly allocates
        // an independent object. Identity, deadlines and tracing are inherited.
        ctx = metrics.WithIndependentRPCStats(ctx)
        resp, err := entry(ctx, req, &grpc.UnaryServerInfo{Server: component, FullMethod:
milvuspb.MilvusService_Search_FullMethodName},
            func(ctx context.Context, req any) (any, error) {
                return component.Search(ctx, req.(*milvuspb.SearchRequest))
            })
        if err != nil {
            return nil, err
        }
        result, ok := resp.(*milvuspb.SearchResults)
        if !ok || result == nil {
            return &milvuspb.SearchResults{Status: merr.Status(merr.WrapErrServiceInternalMsg("Search entry returned an invalid
response"))}, nil
        }
        return result, nil
    }
}
```

Installation happens once. The callback is published atomically after limits and shared channels are initialized. A second installation cannot reset permits while work is active. Before installation, MultiSearch returns a retriable ServiceNotReady status; there is no unchecked Search fallback. Active batches inherit both the request deadline and cancellation of the Proxy component context.

3. Actual dispatch through the existing path

Excerpt from Proxy.MultiSearch, after validation and batch admission. The callback below performs entry checks and invokes ordinary Proxy.Search, which constructs and enqueues its existing SearchTask.

```
// The RPC handler and these bounded workers stay outside the DQL pool.
for range min(parallelism, len(req.Items)) {
    wg.Add(1)
    go func() {
        defer wg.Done()
        for ctx.Err() == nil {
            i := int(next.Add(1) - 1)
            if i >= len(req.Items) {
                return
            }
            select {
            case node.multiSearchActive <- struct{}{}:
            case <-ctx.Done():
                return
            }
            if ctx.Err() != nil {
                <-node.multiSearchActive
                return
            }
            item := req.Items[i]
            // Search and entry interceptors may mutate the request. Never share
            // caller-owned request pointers between workers (or retry attempts).
            search := proto.Clone(item.Search).(*milvuspb.SearchRequest)
            result, err := (*entry)(ctx, search)
            <-node.multiSearchActive
            outcome := &milvuspb.SearchOutcome{Id: item.Id, Result: result}
            if err != nil {
                s := status.Convert(err)
                outcome.Result = nil
                outcome.GrpcCode, outcome.GrpcMessage = int32(s.Code()), s.Message()
            } else if result == nil || result.Status == nil {
                outcome.Result = &milvuspb.SearchResults{Status: merr.Status(merr.WrapErrServiceInternalMsg("Search returned no
result status"))}
            }
            // Reserve protobuf framing overhead as well as outcome bytes.
            if resultBytes.Add(int64(proto.Size(outcome)+16)) > int64(node.multiSearchLimits.MaxResponseBytes) {
                overflow.Store(true)
                cancel()
                return
            }
            out.Items[i] = outcome
        }
    }()
}
wg.Wait()
```

Caller-owned requests are cloned before entry processing. Each child acquires a shared permit until its Search call returns. Item status or transport errors are recorded independently. A deadline or aggregate response-size overflow fails the unary envelope, as before. Permit release does not prove all downstream work has already stopped after cancellation.

4. Validation of the rebuilt binary

Check	Result
Proxy coordination tests, race enabled	12 test functions passed: independent outcomes, validation, pointer/database isolation, shared concurrency/admission, deadlines, response limits, requested concurrency, cancellation while waiting, uninitialized entry, one-time publication, Proxy-context cancellation and publication races.
Real gRPC integration, race enabled	Proxy.MultiSearch receives the RPC. Two children charge exactly 3 + 7 Search tokens; the second is rate-limited while the first succeeds. One subsequent ordinary Search charges five more tokens. The envelope is not charged as a Search. Missing authentication is rejected before execution.
Existing server/stream regressions and isolated statistics	Passed with race detector. All Go checks use dynamic,test and -gcflags='all=-N -l'.
Python SDK tests	Three passed; protocol and SDK code were unchanged by this refactor.
Standalone Docker / 160,000 vectors	All 12 native client checks passed; server reports 3.0.5-multisearch-proxy.

Kubernetes Proxy replica 1	All 12 native client checks passed; server reports 3.0.5-multisearch-proxy.
Kubernetes Proxy replica 2	All 12 native client checks passed; server reports 3.0.5-multisearch-proxy.

The native checks include 50-item equivalence with ordinary Search, independent databases with identical collection names, per-database RBAC, missing/invalid authentication, mixed item failures, envelope limits, deadline and recovery. Kubernetes reuses the earlier 2,048-vector distributed fixture; only the two Proxy replicas were upgraded. QueryNode and other components remain on the preceding v3.0.5 prototype image. Both new Proxy binary hashes match the standalone build.

The new gRPC test initially lacked the access-logger initialization normally performed by server startup. Its fixture was corrected and the final race-enabled run passed. This was a test-fixture failure; no production logging behavior was changed.

---

5. Fresh mixed-traffic measurements on this binary

128 ordinary Search workers first saturate a two-CPU standalone server. A second client then adds 480 logical searches/s, using either eight concurrent Search RPCs or one eight-item MultiSearch RPC. The resident client continues through baseline, injection and recovery. Two repetitions use opposing API order; each run has 5 s warmup, 15 s baseline, 30 s injection and 15 s recovery. Steady-state statistics exclude the first three seconds of each phase.

Added API	Resident baseline QPS	Resident mixed QPS	Resident p99, ms	Injected logical/s	Resident recovery QPS	CPU baseline → mixed
Search	1080.0	583.8	283.68	480.3	1075.5	97.0% → 96.9%
MultiSearch	1088.8	615.2	275.76	480.0	1073.5	97.6% → 96.7%

Compared with ordinary-Search injection, MultiSearch changed resident throughput retention by **+2.44 percentage points** and resident p99 by **-2.8%**. Across four formal runs, **280,665** logical searches completed, including warmup and transitions. Recorded RPC/item errors: **0**; admission drops: **0**; late-generated groups: **0**.

QPS is averaged across repetitions; p99 pools latency samples. CPU includes the whole standalone process. This repeats the resource-contention check on the new binary; it is not a matched before/after measurement proving the ownership refactor improves performance. The earlier stock-v3.0.5 comparison and 14-run mixed-traffic report remain historical evidence for their recorded binary.

The fixture uses eight FLAT collections across two databases, nq=1, topK 5–8, dimensions 128/192/256, L2/IP/COSINE and bounded tmpfs storage. GOMAXPROCS remains four. Benchmark admission limits are 128 shared children/32 batches, above defaults of 64/eight. The shared host and short runs do not establish tenant fairness, production latency, PSC resilience or long-term stability. No confidence intervals are claimed.

6. Build identity and review scope


```
{
  "base": "Milvus v3.0.5 af15c807513eb6508ae22d0f3a6484e8937be1a5",
  "source_parent_commit": "da46e613b189bb26e532eaa1bf436bf1a6cc21bb",
  "source_files_sha256": {
    "internal/proxy/proxy.go": "8d34e3e8d4a09197141d56394a68fb5fe288ee22f2acd07a5fd8b909268c17d4",
    "internal/proxy/multisearch.go": "07275599be703c53c89f9905313412f34a81c383d6f2c2451fd90f6784253e5e",
    "internal/distributed/proxy/service.go": "62904f49398a8204b65a4a29785ea8512d204c95d3e833a275976ebf51de5750",
    "internal/distributed/proxy/multisearch.go": "2d32a443a0049a9fd458ec783f5882cc156f182a02437730a1f29239cc56144f"
  },
  "binary_sha256": "7adcf9232361fad36974f2aaf941bf401233addc9c700bdb08d12e41160ff85a",
  "image": "milvus-multisearch:v305-proxy",
  "image_id": "sha256:61549c1243fde16e277e932c2380f354c58eb42df877a66e48ed19af85cd59b0",
  "build_tags": "dynamic,sonic,with_jemalloc,bytedance_tango",
  "native_libraries": "unchanged from pinned stock v3.0.5 image",
  "go": "go1.26.9",
  "pgo": "configs/pgo/default.pgo is empty",
  "standalone_port": 29564,
  "fixture_vectors": 160000,
  "fixture_databases": 2,
  "fixture_collections": 8,
  "kubernetes_namespace": "multisearch-v305",
  "kubernetes_upgraded_components": "two Proxy replicas only; other components retain preceding v3.0.5 prototype image",
  "kubernetes_fixture_vectors": 2048,
  "performance": {
    "resident_workers": 128,
    "injected_logical_qps": 480,
    "group_size": 8,
    "repetitions": 2,
    "order": "Search then MultiSearch; MultiSearch then Search",
    "cpu_limit": 2,
    "server_cpuset": "64-65",
    "client_cpuset": "32-39",
    "GOMAXPROCS": 4,
    "shared_child_permits": 128,
    "batch_admission_limit": 32,
    "warmup_s": 5,
    "baseline_s": 15,
    "mixed_s": 30,
    "recovery_s": 15,
    "trim_each_phase_s": 3,
    "host_shared": true,
    "storage": "4 GiB tmpfs"
  }
}
```


Go was rebuilt with the release tags using unchanged native libraries from the pinned stock v3.0.5 image. Focused regression tests and native standalone/Kubernetes checks were run; the full repository Go suite and C++/Rust rebuild were not. Unpublished protocol and SDK companion patches are still required. REST, async Python, Cloud gateway/billing validation, full gRPC error details and resumable partial results remain outside this prototype.

The failure-path audit and raw measurements accompany this report. Previous reports are explicitly labeled historical. The PR's primary report now describes this Proxy-owned implementation.

7. Reproduce the measurements

```
# Follow examples/multisearch/README.md to prepare the refactor image and fixture.
.venv/bin/python examples/multisearch/run_refactor_bench.py
.venv/bin/python examples/multisearch/summarize_mixed.py \
  --input-dir artifacts/multisearch-proxy-refactor --server-binary-changed
python examples/multisearch/report_refactor.py
```